

Danmarks
Tekniske
Universitet



Robotics 34753 Project Assignment

AUTHORS

Botond Bencsik - s254257
Emma Royant - s253783
Henrik Maarten Bongers - s253818
Jente Staelens - s252877
Theo Haes - s253792

December 4, 2025

Contents

1	Part 1: Kinematics	1
1.1	Problem 1: Forward Kinematics	1
1.1.1	Robot Stylus Forward Kinematics	2
1.1.2	Robot Camera Forward Kinematics	3
1.2	Problem 2: Inverse Kinematics	4
1.3	Problem 3: Circular Trajectory	7
1.4	Problem 4: Jacobians	9
1.4.1	Derivation of J_{eff} and J_{cam}	9
1.4.2	Numerical Results: J_{eff}	10
1.4.3	Numerical Results: J_{cam}	12
1.5	Problem 5: Joint Velocities	12
1.5.1	Inverse Velocity	13
1.5.2	Interpretation of $\dot{x}_4$	14
2	Part 2: Trajectory Planning	15
2.1	Problem 6: Interpolation polynomials	15
2.2	Problem 7: Path of the end-effector	18
3	Part 3: Singularities and Statics	21
3.1	Problem 8: Condition number of the Jacobian	21
3.1.1	Concept of the Condition Number	21
3.1.2	Results	21
3.2	Problem 9: Joint torques	22
3.2.1	Using Static Force-Torque Relationship	22
4	Part 4: Dynamics	24
4.1	Problem 10: Inverse Dynamics	24
4.1.1	Euler–Lagrange equations of motion	24
4.1.2	Coriolis matrix	26
5	Part 5: Computer Vision and Control	28
5.1	Problem 11: Real World Implementation	28
5.1.1	The Robot Arm	28
5.1.2	Computer Vision	28
5.1.3	Image to Real World Coordinates: Backwards Projection	30

5.1.4	Implementation in Code	34
5.1.5	Results	35
6	Signatures & Formalities	37
6.1	Contributions	37
6.2	Signatures	37
6.3	Use of AI	38
A	Links	40
A.1	Code	40
A.2	Video of Results	40
B	Tables & Numerical Results	41
B.1	Joint configurations for circular trajectory, in $^{\circ}$	41
C	Code	43
C.1	Forward and Inverse Kinematics	43

Chapter 1

Part 1: Kinematics

Contents

1.1	Problem 1: Forward Kinematics	5
1.1.1	Robot Stylus Forward Kinematics	6
1.1.2	Robot Camera Forward Kinematics	7
1.2	Problem 2: Inverse Kinematics	7
1.3	Problem 3: Circular Trajectory	10
1.4	Problem 4: Jacobians	11
1.4.1	Derivation of J_{eff} and J_{cam}	11
1.4.2	Numerical Results: J_{ee}	12
1.4.3	Numerical Results: J_{cam}	13
1.5	Problem 5: Joint Velocities	13
1.5.1	Inverse Velocity	14
1.5.2	Interpretation of $\dot{x}_4$	15

1.1 Problem 1: Forward Kinematics

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Find, by the use of Figure 1.1, the direct kinematic transformations, T_4^0 for the robot stylus, and T_5^0 for the robot camera, as function of all joint angles.

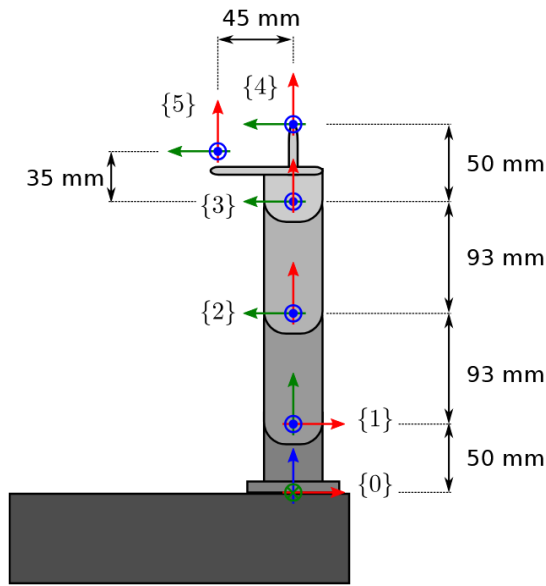


Figure 1.1: Robot Arm and assignment of coordinate frames

Answer:

1.1.1 Robot Stylus Forward Kinematics

The forward kinematics for the robot stylus can be obtained using the Denavit-Hartenberg convention. Tab. 1.1 shows the DH parameters for the coordinate frames in Fig. 1.1, where:

- θ_i = angle from x_{i-1} to x_i (about z_{i-1})
- d_i = distance from o_{i-1} to x_i (along z_{i-1})
- a_i = distance from z_{i-1} to o_i (along x_i)
- α_i = angle from z_{i-1} to z_i (about x_1)

Frame i	θ_i	d_i	a_i	α_i
1	$\theta_1^* = 0$	$d_1 = 50mm$	0	$\alpha_1 = 90^\circ$
2	$\theta_2^* = 90^\circ$	0	$a_2 = 93mm$	0
3	$\theta_3^* = 0$	0	$a_3 = 93mm$	0
4	$\theta_4^* = 0$	0	$a_4 = 50mm$	0

Table 1.1: DH-Parameters up to the stylus

These parameters can then be inserted into the blueprint (Eq. 1.1) for the DH-transformation matrix:

$$T_i^{i-1} = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.1)$$

Thereby the following homogeneous transformation matrices are obtained.

$$T_1^0 = \begin{bmatrix} \cos(\theta_1) & 0 & \sin(\theta_1) & 0 \\ \sin(\theta_1) & 0 & -\cos(\theta_1) & 0 \\ 0 & 1 & 0 & 50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.2)$$

$$T_2^1 = \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) & 0 & 93 \cos(\theta_2) \\ \sin(\theta_2) & \cos(\theta_2) & 0 & 93 \sin(\theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.3)$$

$$T_3^2 = \begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 & 93 \cos(\theta_3) \\ \sin(\theta_3) & \cos(\theta_3) & 0 & 93 \sin(\theta_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.4)$$

$$T_4^3 = \begin{bmatrix} \cos(\theta_4) & -\sin(\theta_4) & 0 & 50 \cos(\theta_4) \\ \sin(\theta_4) & \cos(\theta_4) & 0 & 50 \sin(\theta_4) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.5)$$

From these matrices the mapping to the end-effector can be obtained by:

$$T_4^0 = T_1^0 T_2^1 T_3^2 T_4^3 \quad (1.6)$$

which results in the matrix shown in Eq. 1.7 after simplification using trigonometric identities.

$$T_4^0 = \begin{bmatrix} c_1 c_{234} & -s_{234} c_1 & s_1 & (93 c_2 + 93 c_{23} + 50 c_{234}) c_1 \\ s_1 c_{234} & -s_1 s_{234} & -c_1 & (93 c_2 + 93 c_{23} + 50 c_{234}) s_1 \\ s_{234} & c_{234} & 0 & 93 s_2 + 93 s_{23} + 50 s_{234} + 50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.7)$$

1.1.2 Robot Camera Forward Kinematics

The camera frame does not really follow the DH-conventions. The transformation T_5^4 can be modeled as *translation* from frame 4 to 5 as there is *no difference in their rotation*. Hence:

$$T_5^4 = Trans_x(-15mm) Trans_y(45mm) = \begin{bmatrix} 1 & 0 & 0 & -15 \\ 0 & 1 & 0 & 45 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.8)$$

Then the final transform for the camera can be obtained using the result Eq. 1.7:

$$T_5^0 = T_4^0 T_5^4 \quad (1.9)$$

$$T_5^0 = \begin{bmatrix} c_1 c_{234} & -s_{234} c_1 & s_1 & (93c_2 + 93c_{23} + 35c_{234} - 45s_{234}) c_1 \\ s_1 c_{234} & -s_1 s_{234} & -c_1 & (93c_2 + 93c_{23} + 35c_{234} - 45s_{234}) s_1 \\ s_{234} & c_{234} & 0 & 93s_2 + 93s_{23} + 35s_{234} + 45c_{234} + 50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1.10)$$

1.2 Problem 2: Inverse Kinematics

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Determine the inverse kinematic transformation

$$q = [q_1, q_2, q_3, q_4]^T = f(x_4^0, o_4^0)$$

, where x_4^0 are the first 3 components of the first column of T_4^0 , and o_4^0 are the first 3 components of the last column of T_4^0 , respectively. Satisfy all position components in o_4^0 and only the last component of x_4^0 .

Answer:

The inverse kinematics for this robot arm will be solved using the strategy of kinematic decoupling. The first three joints will be used to satisfy the positional requirement. In other words, they will make sure that o_4^0 is satisfied. The last joint (θ_4) will then be used to satisfy the last component of x_4^0 . For that, first the meaning of the last component of x_4^0 will have to be understood.

The meaning of x_4^0 x_4^0 is the direction, expressed in world/ 0^{th} -frame, of the unit x -vector of the 4^{th} -frame, i.e. of the end-effector. Correspondingly, the last element of that vector shows how much the end-effector points up/down in the world frame.

Given the last element of x_4^0 , θ_{ee} can be deduced. θ_{ee} is the angle of the stylus/end-effector to the world x - y -plane.

$$x_{z4}^0 = \sin(\theta_{ee})$$

Then:

$$\theta_{ee} = \arcsin(x_{z4}^0)$$

From o_x , o_y and o_z this r_{ee} and z_{ee} can be determined (refer to Figs. 1.2 & 1.3).

$$r_{ee}^2 = o_x^2 + o_y^2$$

$$z_{ee} = o_z$$

Then, using our constraint for the end-effector:

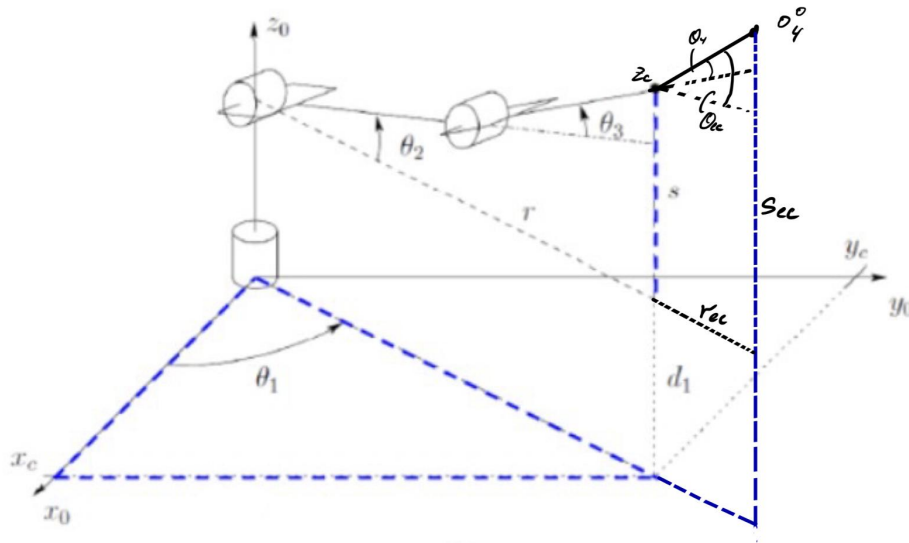


Figure 1.2: Inverse Kinematics Angles

$$r = r_{ee} - a_4 * \cos(\theta_{ee})$$

and

$$z_c = z_{ee} - d_1 - a_4 * \sin(\theta_{ee})$$

can be determined, again refer to Figs. 1.2 & 1.3. So now the distance of the wrist to the world z -axis (r) and the height at which the wrist has to be (z_c) have been determined. The overall rotation of the robot arm (θ_1) can be determined by o_x and o_y by applying basic trigonometry (see Fig. 1.2).

Important Note: This report follows the convention: $\text{atan}(\frac{y}{x}) \rightarrow \text{atan}(y, x)$. This is contrary to the lecture slides and other material of this course. This, however, this is the mainstream convention followed by Python and Matlab.

$$\theta_1 = \text{atan2}(o_y, o_x) \quad (1.11)$$

Now θ_2 and θ_3 must be chosen to satisfy the position of the wrist, defined by r and z_c . For this the trigonometry of Fig. 1.3 will be worked out.

The distance from the base to the tip is given by $\sqrt{r^2 + s^2}$, where

$$s = z_c - d_1 \quad (1.12)$$

Then by applying the law of cosine, where the $\pm$ comes from whether the elbow up or elbow down configuration is modeled :

$$\cos(\pi \pm \theta_3) = \frac{a_2^2 + a_3^2 - r^2 - s^2}{2a_2a_3} \quad (1.13)$$

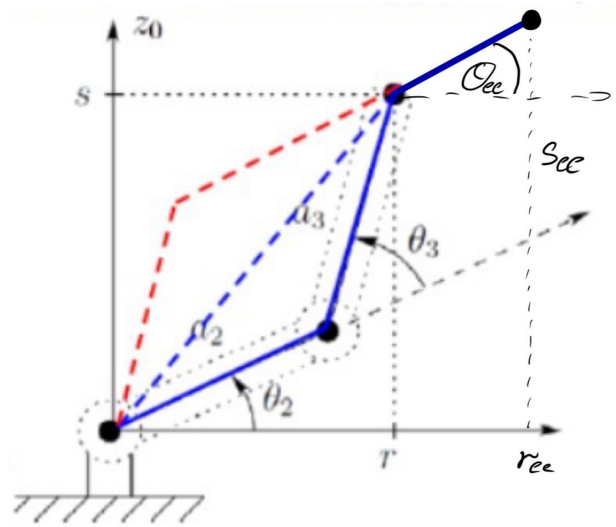


Figure 1.3: Inverse Kinematics Close-Up

Inserting the known values (in *mm*):

$$\cos(\pi + \theta_3^*) = \frac{93^2 + 93^2 - r^2 - s^2}{2 * 93 * 93} \quad (1.14)$$

$$-\cos(\theta_3^*) = \frac{17298 - r^2 - s^2}{17298} \quad (1.15)$$

$$\cos(\theta_3^*) = \frac{r^2 + s^2}{17298} - 1 \quad (1.16)$$

And from identities:

$$\sin(\theta) = \sqrt{1 - \cos^2(\theta)} \quad (1.17)$$

$$\tan(\theta) = \frac{\sin(\theta)}{\cos(\theta)} \quad (1.18)$$

Reminder: please note the different naming convention for arctan!:

$$\theta_3^* = \text{Atan2}(\pm s_3, c_3) = \text{atan2} \left(\pm \sqrt{1 - \left(\frac{r^2 + s^2}{17298} - 1 \right)^2}, \frac{r^2 + s^2}{17298} - 1, \right) \quad (1.19)$$

Then θ_2 can then be determined from trigonometry:

$$\theta_2 = \text{Atan2}(s, r) - \text{Atan2}(a_3 * \sin(\theta_3), a_2 + a_3 * \cos(\theta_3)) \quad (1.20)$$

Then θ_4 can be determined from θ_2 , θ_3 , and θ_{ee} :

$$\theta_4 = -\theta_2 - \theta_3 + \theta_{ee} \quad (1.21)$$

Implementation

These inverse kinematics were implemented in Python and Matlab. The implementations can be found here.

Python

```
robot_arm = Dynamixel()
q = robot_arm.inverse(o_4, x_z0=0)
```

Matlab

```
q = inverseKinematics(o_40, x_z0, d_1, a_2, a_3, a_4)
```

1.3 Problem 3: Circular Trajectory

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

The robot manipulator is now supposed to track, with its stylus tip, a circle with center p_c and radius R . The circle is defined by the equation:

$$p^0(\phi) = p_c^0 + R \begin{bmatrix} 0 \\ \cos(\phi) \\ \sin(\phi) \end{bmatrix}, \quad 0 \leq \phi \leq 2\pi \quad (1.22)$$

Determine a sequence of 37 robot configurations

$$q^{(j)} = \begin{bmatrix} q_1^{(j)} \\ q_2^{(j)} \\ q_3^{(j)} \\ q_4^{(j)} \end{bmatrix} = f(x_4^0 = [\ ?, \ ?, \ 0]^T, o_4^0 = p^0(\phi_j)), \quad j = 0, 1, \dots, 36$$

such that the stylus tip tracks 36 equidistant points along the circle while remaining horizontal throughout the motion.

Answer

In this exercise, a circular path for the end-effector was generated in the y - z plane, centered at

$$p_c = \begin{bmatrix} 150 \\ 0 \\ 120 \end{bmatrix} \text{ mm}, \quad R = 32 \text{ mm}.$$

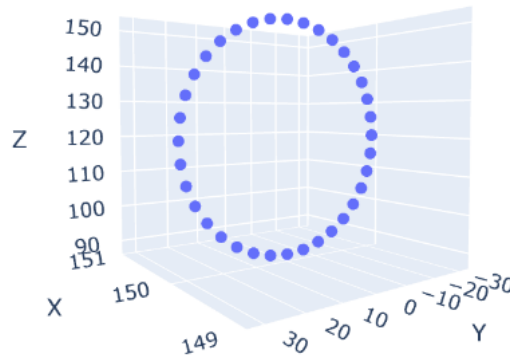


Figure 1.4: 3D-Trajectory

The circle was discretized into 37 equally spaced points between $\phi_0 = 0$ and $\phi_{36} = 2\pi$. For each point on the circle, the corresponding end-effector position o_4^0 was computed as:

$$o_4^0 = p_c + \begin{bmatrix} 0 \\ R \cos(\phi_j) \\ R \sin(\phi_j) \end{bmatrix}$$

The stylus orientation was kept horizontal by constraining the z -component of the end-effector x -axis, x_z^0 , to zero.

For each target point o_4^0 , the inverse kinematics function derived in Problem 2 was used to compute the corresponding joint configuration:

$$q^{(j)} = \text{inverseKinematics}(o_4^0, x_z^0 = 0, d_1, a_2, a_3, a_4)$$

Here:

- `R` and `pc` define the circle geometry.
- `phi` contains 37 equally spaced angular positions along the circle.
- `o04` is the desired end-effector position at each step.
- `inverseKinematics` computes the joint angles $[q_1, q_2, q_3, q_4]^T$ ensuring the stylus tip remains horizontal.

The resulting matrix `q_seq` contains 37 sets of joint angles that enable the robot stylus to trace a smooth circular trajectory in space. The array `P` stores the corresponding end-effector positions along the path.

The resulting joint configurations can be found in B.1.

A video of the simulated robot configurations can be seen [here](#).

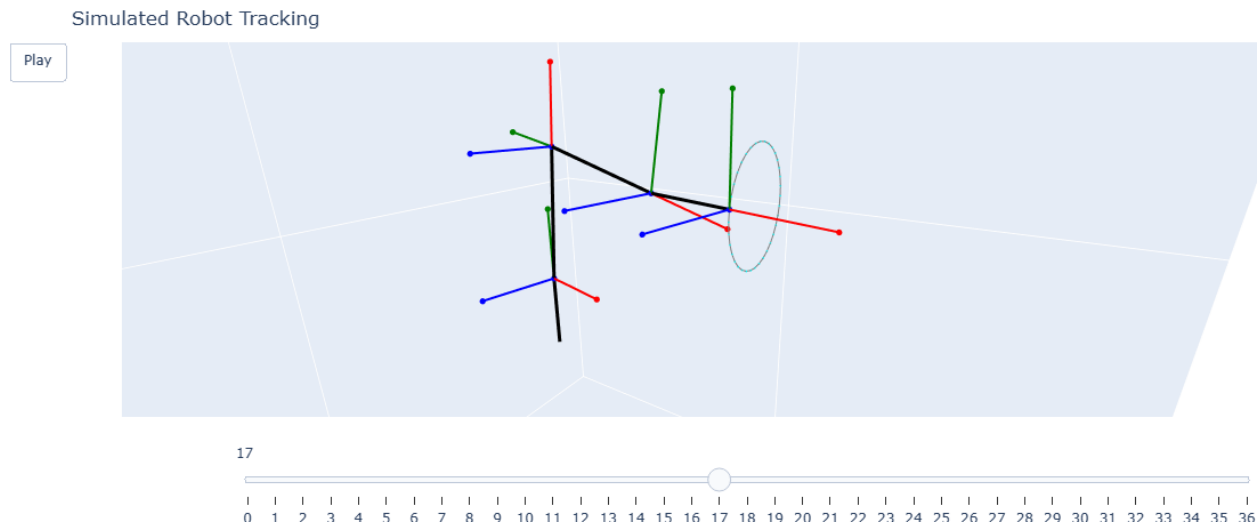


Figure 1.5: Interactive 3D-Visualization of tracking the circular trajectory

1.4 Problem 4: Jacobians

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Determine the Jacobian of the manipulator for the robot end-effector and the Jacobian for the robot camera (as a function of the joint configuration q). Report the numerical results for the two Jacobians at $\phi = 0$, $\phi = \pi/2$, $\phi = \pi$, and $\phi = 3\pi/2$ along the path studied in Problem 3.

Answer

1.4.1 Derivation of J_{eff} and J_{cam}

In this problem, two Jacobians were computed:

- J_{eff} : the Jacobian of the stylus end-effector, relating joint velocities $\dot{q}$ to the linear and angular velocity of the stylus tip,

$$\begin{bmatrix} \dot{p}_{eff} \\ \omega_{eff} \end{bmatrix} = J_{eff}(q) \dot{q}$$

- J_{cam} : the Jacobian of the camera, expressing the same relationship for the camera reference frame attached to the end-effector.

$$\begin{bmatrix} \dot{p}_{cam} \\ \omega_{cam} \end{bmatrix} = J_{cam}(q) \dot{q}$$

The computation follows the standard geometric Jacobian formulation for revolute

joints, where for each joint i :

$$J_{v_i} = z_{i-1} \times (o_n - o_{i-1}), \quad J_{\omega_i} = z_{i-1}$$

with z_{i-1} and o_{i-1} representing, respectively, the z -axis and origin of frame $\{i-1\}$ expressed in the base frame. o_n is the origin of the point whose velocities are computed. In this exercise that are o_{eff} and o_{cam} , respectively.

The origin $o_0 = [0 \ 0 \ 0]^T$ and the transformation matrices T_1^0 , T_2^0 and T_3^0 can be used to find the o_{i-1} 's (4th column, rows 1 to 3) and z_{i-1} 's o_{i-1} 's (3rd column, rows 1 to 3), and T_4^0 and T_5^0 can be used to compute o_n , for J_{eff} and J_{cam} , respectively.

This gives the following blueprints for the Jacobians:

$$J_{\text{eff/cam}} = \begin{bmatrix} z_0 \times (o_{4/5} - o_0) & z_1 \times (o_{4/5} - o_1) & z_2 \times (o_{4/5} - o_2) & z_3 \times (o_{4/5} - o_3) \\ z_0 & z_1 & z_2 & z_3 \end{bmatrix} \quad (1.23)$$

This results in:

$$J_{\text{eff}} = \begin{bmatrix} -(93c_2 + 93c_{23} + 50c_{234})s_1 & -(93s_2 + 93s_{23} + 50s_{234})c_1 & -(93s_{23} + 50s_{234})c_1 & -50s_{234}c_1 \\ (93c_2 + 93c_{23} + 50c_{234})c_1 & -(93s_2 + 93s_{23} + 50s_{234})s_1 & -(93s_{23} + 50s_{234})s_1 & -50s_1s_{234} \\ 0 & 93c_2 + 93c_{23} + 50c_{234} & 93c_{23} + 50c_{234} & 50c_{234} \\ 0 & s_1 & s_1 & s_1 \\ 0 & -c_1 & -c_1 & -c_1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad (1.24)$$

$$J_{\text{cam}} = \begin{bmatrix} (45s_{234} - 93c_2 - 93c_{23} - 35c_{234})s_1 & -(93s_2 + 93s_{23} + 35s_{234} + 45c_{234})c_1 & -(93s_{23} + 35s_{234} + 45c_{234})c_1 & -5(7s_{234} + 9c_{234})c_1 \\ (-45s_{234} + 93c_2 + 93c_{23} + 35c_{234})c_1 & -(93s_2 + 93s_{23} + 35s_{234} + 45c_{234})s_1 & -(93s_{23} + 35s_{234} + 45c_{234})s_1 & -5(7s_{234} + 9c_{234})s_1 \\ 0 & -45s_{234} + 93c_2 + 93c_{23} + 35c_{234} & -45s_{234} + 93c_{23} + 35c_{234} & -45s_{234} + 35c_{234} \\ 0 & s_1 & s_1 & s_1 \\ 0 & -c_1 & -c_1 & -c_1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

To solve this Matlab and Python functions have been written. The substitution of the numerical joint configuration q into the symbolic matrices was performed using the `subs()` function, and the result was evaluated numerically using `double()`.

Here:

- `computeJacobians` computes both the end-effector and camera Jacobians for a given joint configuration.
- The indices `i1-i4` correspond to the four positions on the circular trajectory.
- `q_selected` stores the corresponding joint configurations.
- `J` contains the computed Jacobian matrices for the stylus tip.

The resulting Jacobians provide the linear and angular velocity mappings for both the stylus and camera frames at the four specified positions along the trajectory.

1.4.2 Numerical Results: J_{eff}

These jacobians will calculate the linear velocities in $\frac{\text{mm}}{\text{s}}$ and angular velocities in $\frac{\text{rad}}{\text{s}}$!

$$\phi = 0^\circ$$

$$\begin{bmatrix} -32.0 & -69.0 & 22.0 & 0 \\ 1.5 \cdot 10^2 & -15.0 & 4.6 & 0 \\ 0 & 1.5 \cdot 10^2 & 1.4 \cdot 10^2 & 50.0 \\ 0 & 0.21 & 0.21 & 0.21 \\ 0 & -0.98 & -0.98 & -0.98 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.25)$$

$$\phi = 90^\circ$$

$$\begin{bmatrix} 0 & -1.0 \cdot 10^2 & -9.3 & 0 \\ 1.5 \cdot 10^2 & 0 & 0 & 0 \\ 0 & 1.5 \cdot 10^2 & 1.4 \cdot 10^2 & 50.0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.0 & -1.0 & -1.0 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.26)$$

$$\phi = 180^\circ$$

$$\begin{bmatrix} 32.0 & -69.0 & 22.0 & 0 \\ 1.5 \cdot 10^2 & 15.0 & -4.6 & 0 \\ 0 & 1.5 \cdot 10^2 & 1.4 \cdot 10^2 & 50.0 \\ 0 & -0.21 & -0.21 & -0.21 \\ 0 & -0.98 & -0.98 & -0.98 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.27)$$

$$\phi = 270^\circ$$

$$\begin{bmatrix} 0 & -38.0 & 52.0 & 0 \\ 1.5 \cdot 10^2 & 0 & 0 & 0 \\ 0 & 1.5 \cdot 10^2 & 1.3 \cdot 10^2 & 50.0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.0 & -1.0 & -1.0 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.28)$$

1.4.3 Numerical Results: J_{cam}

$$\phi = 0^\circ$$

$$\begin{bmatrix} -28.9 & -112.0 & -22.4 & -44.0 \\ 135.0 & -24.0 & -4.78 & -9.39 \\ 0 & 138.0 & 125.0 & 35.0 \\ 0 & 0.209 & 0.209 & 0.209 \\ 0 & -0.978 & -0.978 & -0.978 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.29)$$

$$\phi = 90^\circ$$

$$\begin{bmatrix} 0 & -147.0 & -54.3 & -45.0 \\ 135.0 & 0 & 0 & 0 \\ 0 & 135.0 & 128.0 & 35.0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.0 & -1.0 & -1.0 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.30)$$

$$\phi = 180^\circ$$

$$\begin{bmatrix} 28.9 & -112.0 & -22.4 & -44.0 \\ 135.0 & 24.0 & 4.78 & 9.39 \\ 0 & 138.0 & 125.0 & 35.0 \\ 0 & -0.209 & -0.209 & -0.209 \\ 0 & -0.978 & -0.978 & -0.978 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.31)$$

$$\phi = 270^\circ$$

$$\begin{bmatrix} 0 & -83.0 & 7.12 & -45.0 \\ 135.0 & 0 & 0 & 0 \\ 0 & 135.0 & 112.0 & 35.0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.0 & -1.0 & -1.0 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.32)$$

1.5 Problem 5: Joint Velocities

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Compute the joint velocities $\dot{q}$ at $\varphi = \frac{\pi}{2}$, along the path from Problem 3, so that the stylus tip velocity is $v_4^0 = [0, -3, 0]$ mm/s and $\dot{x}_4 = [?, ?, 0]$. Hint: the last quantity $\dot{x}_4$, you have not seen it as such in the course before, so you need to think about how to interpret it in terms of angular velocities.

Answer

1.5.1 Inverse Velocity

The inverse velocity problem can be solved using the following equation, where b is an arbitrary vector [3, pp.139-141]:

$$\dot{q} = J^+ \xi + (I_{n \times n} - J^+ J) b_{n \times 1} \quad (1.33)$$

However, if the goal is to minimize the joint velocities b can be set to zero. For the proof refer to [3, pp.139-141].

$$\dot{q} = J^+ \xi \quad (1.34)$$

As the robot arm in question is underactuated, meaning there are more degrees of freedom than joints ($m < n$). J^+ is the right hand pseudo inverse of the Jacobian $J = J_{eff}$.

$$J^+ = [J^T J]^{-1} J^T \quad (1.35)$$

For $\varphi = \frac{\pi}{2}$ the desired coordinates in mm are $[150 \ 0 \ 152]^T$. Solving the inverse kinematics for gives:

- $\theta_1 = 0^\circ$
- $\theta_2 = 85.40^\circ$
- $\theta_3 = -79.66^\circ$
- $\theta_4 = -5.74^\circ$

Substituting into Eq. 1.24:

$$J_{eff} = \begin{bmatrix} 0 & -102.0 & -9.3002 & 0 \\ 150.0 & 0 & 0 & 0 \\ 0 & 150.0 & 142.53 & 50.0 \\ 0 & 0 & 0 & 0 \\ 0 & -1.0 & -1.0 & -1.0 \\ 1.0 & 0 & 0 & 0 \end{bmatrix} \quad (1.36)$$

Obtaining its pseudo inverse with Eq. 1.35:

$$J_{eff}^+ = \begin{bmatrix} 0 & 0.0066664 & 0 & 0 & 0 & 4.4442 \cdot 10^{-5} \\ -0.010879 & 0 & -0.0010986 & 0 & -0.054715 & 0 \\ 0.011803 & 0 & 0.011963 & 0 & 0.60008 & 0 \\ -0.00099182 & 0 & -0.011108 & 0 & -1.5465 & 0 \end{bmatrix} \quad (1.37)$$

Defining the desired ξ based on v_4^0 :

$$\xi_{des} = \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (1.38)$$

Then:

$$\dot{q} = J^+ \xi = \begin{bmatrix} 0 & 0.0066664 & 0 & 0 & 0 & 4.4442 \cdot 10^{-5} \\ -0.010879 & 0 & -0.0010986 & 0 & -0.054715 & 0 \\ 0.011803 & 0 & 0.011963 & 0 & 0.60008 & 0 \\ -0.00099182 & 0 & -0.011108 & 0 & -1.5465 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.02 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (1.39)$$

Now the actual ξ_{actual} can be obtained with the calculated $\dot{q}$.

$$\xi_{actual} = J_{eff} \dot{q} = \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ 0 \\ -0.02 \end{bmatrix} \quad (1.40)$$

1.5.2 Interpretation of $\dot{x}_4$

$\dot{x}_4 = [?, ?, 0]$ can be interpreted as the time-derivative of the x -vector of the 4th coordinate frame / the end-effectors frame. As seen in the calculated $\dot{q}$ in Eq. 1.39, only the first servo motor moves. This means, that if anything rotates it will rotate around the world z -axis. Furthermore, the inverse kinematics solution constrains the x -vector to lie in the xy -plane. This explains why the z -component of $\dot{x}_4$ is equal to 0. Anything else would violate the inverse kinematics. Also the last three components of ξ_{actual} indicate that the end effector is yawing/rotating around the global z -axis. The first two components of $\dot{x}_4$ will be non-zero as a result of that angular velocity around the world z -axis.

Chapter 2

Part 2: Trajectory Planning

Contents

2.1	Problem 6: Interpolation polynomials	17
2.2	Problem 7: Path end-effector	19

2.1 Problem 6: Interpolation polynomials

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Use the inverse computed joint configurations $q^{(0)}, q^{(9)}, q^{(18)}, q^{(27)}, q^{(36)}$ from Problem 3, to find suitable interpolation polynomials for the following segments:

Segment A: $q^{(0)} \rightarrow q^{(9)}, 0 \leq t_A \leq 2s$

$$q_1(t_A) = A_{15}t_A^5 + A_{14}t_A^4 + A_{13}t_A^3 + A_{12}t_A^2 + A_{11}t_A + A_{10}$$

$$q_2(t_A) = A_{25}t_A^5 + A_{24}t_A^4 + A_{23}t_A^3 + A_{22}t_A^2 + A_{21}t_A + A_{20}$$

$$q_3(t_A) = A_{35}t_A^5 + A_{34}t_A^4 + A_{33}t_A^3 + A_{32}t_A^2 + A_{31}t_A + A_{30}$$

$$q_4(t_A) = A_{45}t_A^5 + A_{44}t_A^4 + A_{43}t_A^3 + A_{42}t_A^2 + A_{41}t_A + A_{40}$$

Segment B: $q^{(9)} \rightarrow q^{(18)}, 0 \leq t_B \leq 2s$

$$q_1(t_B) = B_{15}t_B^5 + B_{14}t_B^4 + B_{13}t_B^3 + B_{12}t_B^2 + B_{11}t_B + B_{10}$$

$$q_2(t_B) = B_{25}t_B^5 + B_{24}t_B^4 + B_{23}t_B^3 + B_{22}t_B^2 + B_{21}t_B + B_{20}$$

$$q_3(t_B) = B_{35}t_B^5 + B_{34}t_B^4 + B_{33}t_B^3 + B_{32}t_B^2 + B_{31}t_B + B_{30}$$

$$q_4(t_B) = B_{45}t_B^5 + B_{44}t_B^4 + B_{43}t_B^3 + B_{42}t_B^2 + B_{41}t_B + B_{40}$$

Segment C: $q^{(18)} \rightarrow q^{(27)}, 0 \leq t_C \leq 2s$

$$q_1(t_C) = C_{15}t_C^5 + C_{14}t_C^4 + C_{13}t_C^3 + C_{12}t_C^2 + C_{11}t_C + C_{10}$$

$$q_2(t_C) = C_{25}t_C^5 + C_{24}t_C^4 + C_{23}t_C^3 + C_{22}t_C^2 + C_{21}t_C + C_{20}$$

$$q_3(t_C) = C_{35}t_C^5 + C_{34}t_C^4 + C_{33}t_C^3 + C_{32}t_C^2 + C_{31}t_C + C_{30}$$

$$q_4(t_C) = C_{45}t_C^5 + C_{44}t_C^4 + C_{43}t_C^3 + C_{42}t_C^2 + C_{41}t_C + C_{40}$$

Segment D: $q^{(27)} \rightarrow q^{(36)}, 0 \leq t_D \leq 2s$

$$q_1(t_D) = D_{15}t_D^5 + D_{14}t_D^4 + D_{13}t_D^3 + D_{12}t_D^2 + D_{11}t_D + D_{10}$$

$$q_2(t_D) = D_{25}t_D^5 + D_{24}t_D^4 + D_{23}t_D^3 + D_{22}t_D^2 + D_{21}t_D + D_{20}$$

$$q_3(t_D) = D_{35}t_D^5 + D_{34}t_D^4 + D_{33}t_D^3 + D_{32}t_D^2 + D_{31}t_D + D_{30}$$

$$q_4(t_D) = D_{45}t_D^5 + D_{44}t_D^4 + D_{43}t_D^3 + D_{42}t_D^2 + D_{41}t_D + D_{40}$$

Determine the coefficients $A_{ij}, B_{ij}, C_{ij}, D_{ij}$ so that

$$v(t_A = 0) = [0, 0, 0]^T \text{ mm/s} \quad \ddot{q}(t_A = 0) = [0, 0, 0, 0]^T \text{ rad/s}^2$$

$$v(t_A = 2) = v(t_B = 0) = [0, -27, 0]^T \text{ mm/s} \quad \ddot{q}(t_A = 2) = \ddot{q}(t_B = 0) = [0, 0, 0, 0]^T \text{ rad/s}^2$$

$$v(t_B = 2) = v(t_C = 0) = [0, 0, -27]^T \text{ mm/s} \quad \ddot{q}(t_B = 2) = \ddot{q}(t_C = 0) = [0, 0, 0, 0]^T \text{ rad/s}^2$$

$$v(t_C = 2) = v(t_D = 0) = [0, 27, 0]^T \text{ mm/s} \quad \ddot{q}(t_C = 2) = \ddot{q}(t_D = 0) = [0, 0, 0, 0]^T \text{ rad/s}^2$$

$$v(t_D = 2) = [0, 0, 0]^T \text{ mm/s} \quad \ddot{q}(t_D = 2) = [0, 0, 0, 0]^T \text{ rad/s}^2$$

Answer

The procedure for this problem is identical for each of the four segments. Therefore, the approach for segment A will be discussed, after which the solution for all four segments will be presented.

This problem can be described as a set of four equations with each six unknowns. The given boundary conditions are $q(0), \ddot{q}(0), v(0), q(2), \ddot{q}(2), v(2)$. Using the solution of the inverse velocity problem derived in problem 5, the boundary conditions on v can be transformed to boundary conditions on $\dot{q}$. Hereby, it is assumed that the end effector is horizontal at each knot-point. After this manipulation, the boundary conditions are expressed as values for $q(0), \dot{q}(0), \ddot{q}(0), q(2), \dot{q}(2), \ddot{q}(2)$. These combine to 24 boundary conditions in total, for as many unknowns.

In order to use the boundary conditions on $\dot{q}$ and $\ddot{q}$, the four equations need to be differentiated respectively once and twice with respect to time. When evaluating this set of twelve equations at both $t_A = 0s$ and $t_A = 2s$, the problem is reduced to the one of solving a set of 24 linear equations in the unknown variables A_{ij} . These equations are written in 2.1 and can be solved numerically. The numerical values for $A_{ij}, B_{ij}, C_{ij}, D_{ij}$ are shown in equations 2.2 to 2.5.

$$\begin{bmatrix}
t_0^5 & t_0^4 & t_0^3 & t_0^2 & t_0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & t_0^5 & t_0^4 & t_0^3 & t_0^2 & t_0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & t_0^5 & t_0^4 & t_0^3 & t_0^2 & t_0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & t_0^5 & t_0^4 & t_0^3 & t_0^2 & t_0 & 1 \\
t_1^5 & t_1^4 & t_1^3 & t_1^2 & t_1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & t_1^5 & t_1^4 & t_1^3 & t_1^2 & t_1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & t_1^5 & t_1^4 & t_1^3 & t_1^2 & t_1 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & t_1^5 & t_1^4 & t_1^3 & t_1^2 & t_1 & 1 \\
5t_0^4 & 4t_0^3 & 3t_0^2 & 2t_0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 5t_0^4 & 4t_0^3 & 3t_0^2 & 2t_0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5t_0^4 & 4t_0^3 & 3t_0^2 & 2t_0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5t_0^4 & 4t_0^3 & 3t_0^2 & 2t_0 & 1 & 0 \\
5t_1^4 & 4t_1^3 & 3t_1^2 & 2t_1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 5t_1^4 & 4t_1^3 & 3t_1^2 & 2t_1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5t_1^4 & 4t_1^3 & 3t_1^2 & 2t_1 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 5t_1^4 & 4t_1^3 & 3t_1^2 & 2t_1 & 1 & 0 \\
20t_0^3 & 12t_0^2 & 6t_0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 20t_0^3 & 12t_0^2 & 6t_0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 20t_0^3 & 12t_0^2 & 6t_0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 20t_0^3 & 12t_0^2 & 6t_0 & 2 & 0 & 0 \\
20t_1^3 & 12t_1^2 & 6t_1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 20t_1^3 & 12t_1^2 & 6t_1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 20t_1^3 & 12t_1^2 & 6t_1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 20t_1^3 & 12t_1^2 & 6t_1 & 2 & 0 & 0
\end{bmatrix}
\begin{matrix}
A_{15} \\ A_{14} \\ A_{13} \\ A_{12} \\ A_{11} \\ A_{10} \\ A_{25} \\ A_{24} \\ A_{23} \\ A_{22} \\ A_{21} \\ A_{20} \\ A_{35} \\ A_{34} \\ A_{33} \\ A_{32} \\ A_{31} \\ A_{30} \\ A_{45} \\ A_{44} \\ A_{43} \\ A_{42} \\ A_{41} \\ A_{40}
\end{matrix}
=
\begin{bmatrix}
q_1(t_0) \\ q_2(t_0) \\ q_3(t_0) \\ q_4(t_0) \\ q_1(t_1) \\ q_2(t_1) \\ q_3(t_1) \\ q_4(t_1) \\ \dot{q}_1(t_0) \\ \dot{q}_2(t_0) \\ \dot{q}_3(t_0) \\ \dot{q}_4(t_0) \\ \dot{q}_1(t_1) \\ \dot{q}_2(t_1) \\ \dot{q}_3(t_1) \\ \dot{q}_4(t_1) \\ \ddot{q}_1(t_0) \\ \ddot{q}_2(t_0) \\ \ddot{q}_3(t_0) \\ \ddot{q}_4(t_0) \\ \ddot{q}_1(t_1) \\ \ddot{q}_2(t_1) \\ \ddot{q}_3(t_1) \\ \ddot{q}_4(t_1)
\end{bmatrix}
\quad (2.1)$$

$$A = \begin{bmatrix}
-0.0057 & 0.0396 & -0.0827 & 0.0000 & 0.0000 & 0.2102 \\
0.0113 & -0.0565 & 0.0753 & 0.0000 & 0.0000 & 1.4302 \\
0.0524 & -0.2622 & 0.3496 & 0.0000 & 0.0000 & -1.6699 \\
-0.0637 & 0.3187 & -0.4249 & 0.0000 & 0.0000 & 0.2397
\end{bmatrix} \quad (2.2)$$

$$B = \begin{bmatrix}
-0.0057 & 0.0171 & 0.0073 & 0.0000 & -0.1800 & 0.0000 \\
0.0017 & -0.0041 & -0.0060 & 0.0000 & 0.0000 & 1.4904 \\
-0.0113 & 0.0700 & -0.1300 & 0.0000 & 0.0000 & -1.3903 \\
0.0096 & -0.0659 & 0.1360 & 0.0000 & -0.0000 & -0.1002
\end{bmatrix} \quad (2.3)$$

$$C = \begin{bmatrix}
0.0057 & -0.0396 & 0.0827 & 0.0000 & -0.0000 & -0.2102 \\
-0.0075 & 0.0329 & -0.0323 & 0.0000 & -0.0693 & 1.4302 \\
-0.0050 & 0.0111 & 0.0217 & 0.0000 & -0.2196 & -1.6699 \\
0.0124 & -0.0441 & 0.0106 & 0.0000 & 0.2889 & 0.2397
\end{bmatrix} \quad (2.4)$$

$$D = \begin{bmatrix}
0.0057 & -0.0171 & -0.0073 & 0.0000 & 0.1800 & -0.0000 \\
0.0204 & -0.1022 & 0.1363 & 0.0000 & -0.0000 & 1.3212 \\
0.0461 & -0.2307 & 0.3077 & 0.0000 & -0.0000 & -1.9160 \\
-0.0666 & 0.3329 & -0.4439 & 0.0000 & 0.0000 & 0.5949
\end{bmatrix} \quad (2.5)$$

2.2 Problem 7: Path of the end-effector

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Plot the actual path of the end-effector for the entire period from $t = 0$ to 8 s for the interpolated trajectory from problem 6 and compare it to the exact desired circular path. Try to improve the approximation, either by using more knot-points or by using different interpolation functions than those found in problem 6.

Answer

Figure 2.1 shows the comparison between the actual and the computed interpolated circular path of problem 6. The interpolation can be improved by adding more interpolation points. This is done in 2.2, where the same fifth order interpolation polynomials and similar boundary conditions were used, but for 40 knot-points. By increasing the number of knot-points by a factor 10, the maximum deviation was reduced to $12 \mu\text{m}$, which is approximately a factor 100 smaller than with 4 knot-points.

It can be noted that the choice of putting the joint angle acceleration to zero on each knot-point is rather interesting. Indeed, in order to describe a circular path, the acceleration of the end-effector needs to point to the center of the circle. This acceleration results in non-zero joint accelerations. Replacing the boundary conditions on the joint accelerations by the necessary values to describe the circular path would improve the interpolation. This was however not implemented as an assistant told us that we were not allowed to change the boundary conditions on the joint accelerations.

The other option to improve the interpolation is to change the interpoland. A different order interpolation polynomial, by preference with an uneven order so that symmetric boundary conditions can be imposed, could be used. However, figure 2.3 suggest that the joint angles vary sinusoidal, such that the following set of functions could be used as interpolands instead:

$$\begin{aligned} q_1(t) &= A_1 \sin(\omega t + \phi_1) \\ q_2(t) &= A_2 \sin(\omega t + \phi_2) \\ q_3(t) &= A_3 \sin(\omega t + \phi_3) \\ q_4(t) &= A_4 \sin(\omega t + \phi_4) \end{aligned} \tag{2.6}$$

Higher frequency terms could be added, as shown in 2.7, to refine the interpolation in case the joint angles contain higher frequencies or in case the boundary conditions on the acceleration need to be fulfilled. However, this interpoland is very specific to this use case and has few real world applications and was therefore not calculated.

$$\begin{aligned} q_1(t) &= A_{11} \sin(\omega t + \phi_{11}) + A_{12} \sin(2\omega t + \phi_{12}) + \dots \\ q_2(t) &= A_{21} \sin(\omega t + \phi_{21}) + A_{22} \sin(2\omega t + \phi_{22}) + \dots \\ q_3(t) &= A_{31} \sin(\omega t + \phi_{31}) + A_{32} \sin(2\omega t + \phi_{32}) + \dots \\ q_4(t) &= A_{41} \sin(\omega t + \phi_{41}) + A_{42} \sin(2\omega t + \phi_{42}) + \dots \end{aligned} \tag{2.7}$$

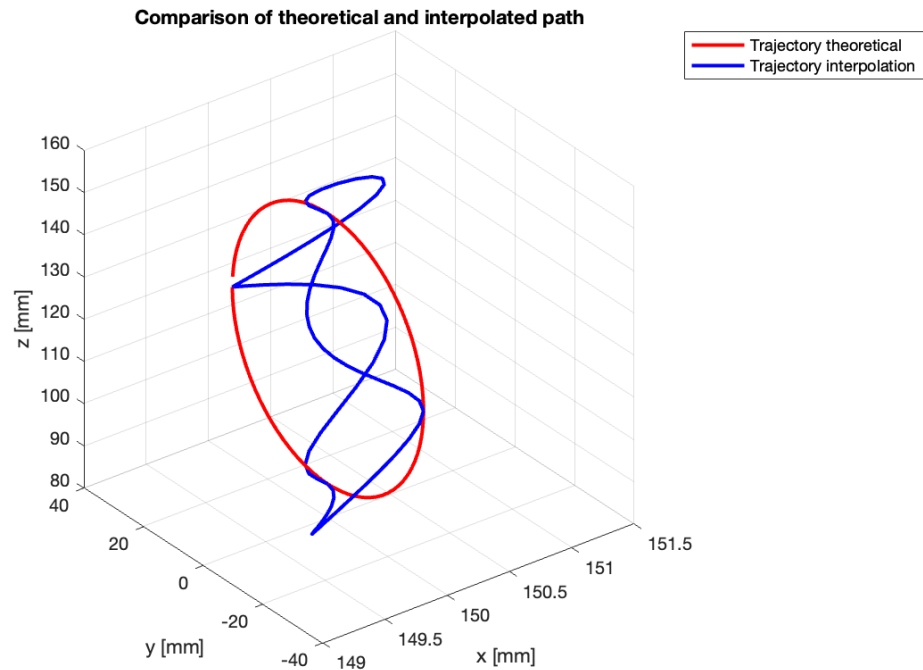


Figure 2.1: Actual vs interpolated trajectory of Problem 6, using 4 interpolation points

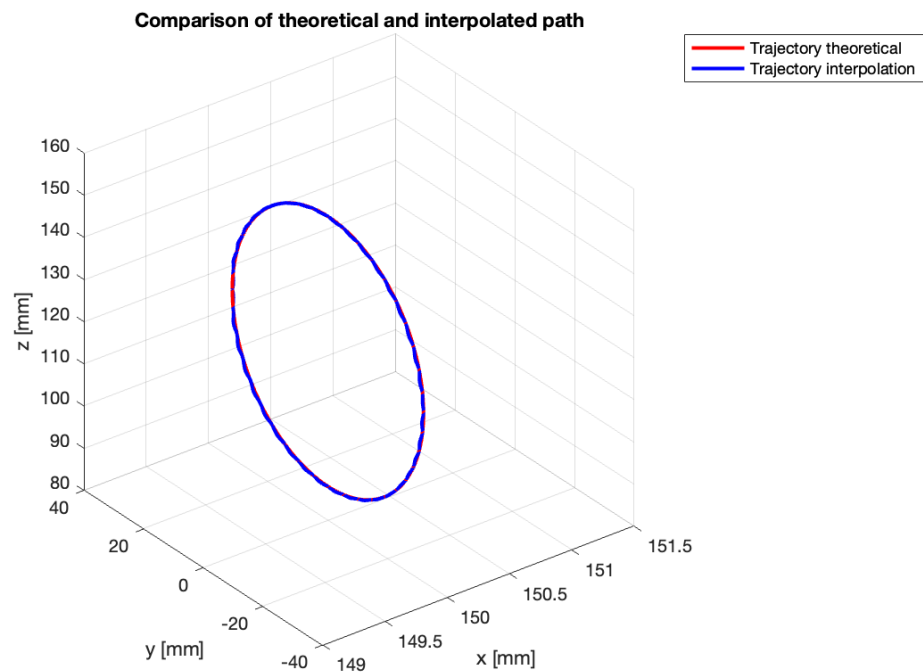


Figure 2.2: Actual vs interpolated trajectory of Problem 6, using 40 interpolation points

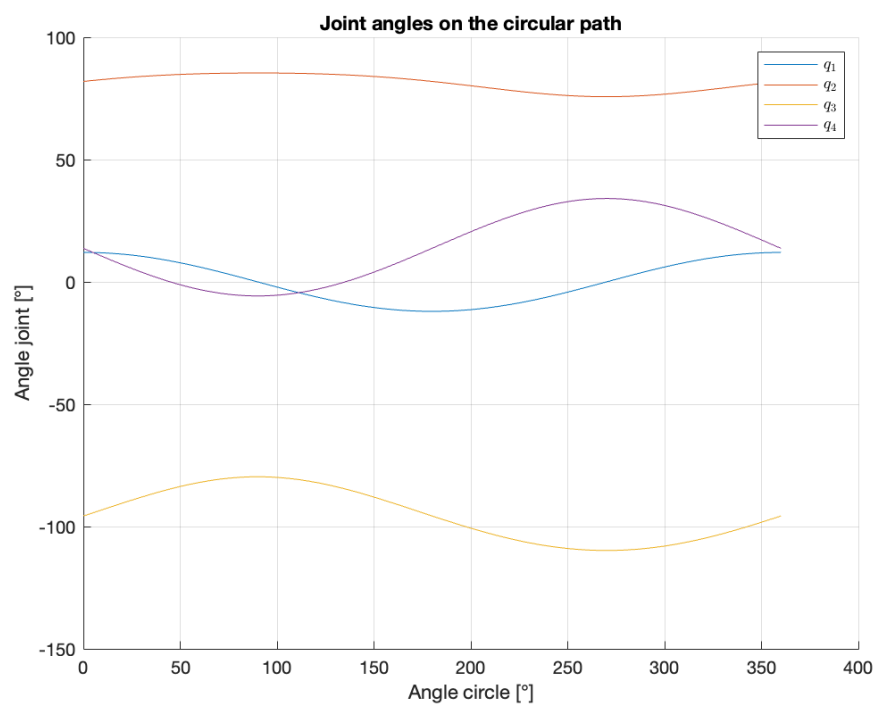


Figure 2.3: Theoretical joint angles on circular path

Chapter 3

Part 3: Singularities and Statics

Contents

3.1	Problem 8: Condition number of the Jacobian	23
3.1.1	Concept of the Condition Number	23
3.1.2	Results	23
3.2	Problem 9: Joint torques	23
3.2.1	Static Force-Torque Relationship	24
3.2.2	Results and Interpretation	24

3.1 Problem 8: Condition number of the Jacobian

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Plot the condition number of the Jacobian matrix of the manipulator along the path from Problem 3 as well as along the actual path from Problem 6 or 7, and evaluate if the path includes any singularities.

3.1.1 Concept of the Condition Number

The condition number $C(\mathbf{J})$ of the Jacobian matrix $\mathbf{J}$ is defined as the ratio of the largest to the smallest singular value:

$$C(\mathbf{J}) = \frac{\sigma_{\max}(\mathbf{J})}{\sigma_{\min}(\mathbf{J})}$$

3.1.2 Results

Fig. 3.1 shows the condition number computed for the theoretical, perfectly circular, trajectory and the interpolated trajectory. A singularity would be indicated by an infinite condition number. However, as seen in the figure below, for both trajectories, the condition

number is not infinite, which means that the trajectory is actually realizable, in terms of joint velocities.

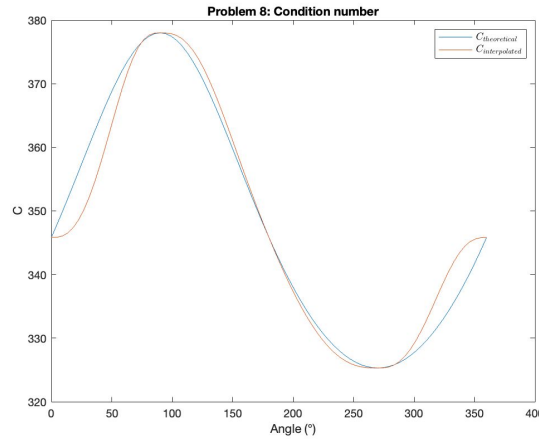


Figure 3.1: Condition numbers for both paths

3.2 Problem 9: Joint torques

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Neglecting the own mass of the robot arm, and assuming a weight of 1 N acting on the end-effector along the negative z_0 direction, calculate and plot all joint torques $\tau_1, \tau_2, \tau_3, \tau_4$ as a function of the position $\phi \in [0, 2\pi]$. Hint: Neglect friction and other losses.

Answer:

3.2.1 Using Static Force-Torque Relationship

The required joint torques ($\boldsymbol{\tau}$) to balance an external force ($\mathbf{F}_{ee}$) applied at the end-effector are determined by the static force-torque relationship derived from the principle of virtual work:

$$\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}_{ee}$$

For this problem, where the load is a pure force ($\mathbf{F}_{ee} = [0; 0; -1]^T$ N) and no external moment is applied, only the translational Jacobian $\mathbf{J}_v$ is used: $\boldsymbol{\tau} = \mathbf{J}_v^T \mathbf{F}_{ee}$.

Plotting Results and Interpretation

The joint torques results are plotted in Fig. 3.2 shown below. These are the joint torques caused by the externally applied force. To keep the robot arm static, their negative would have to be applied in each joint. $\tau_1 = 0$ as the force only acts in the world z -axis and as this is the only axis the first motor can exert a torque around, it is not able to generate an upward force in that axis.

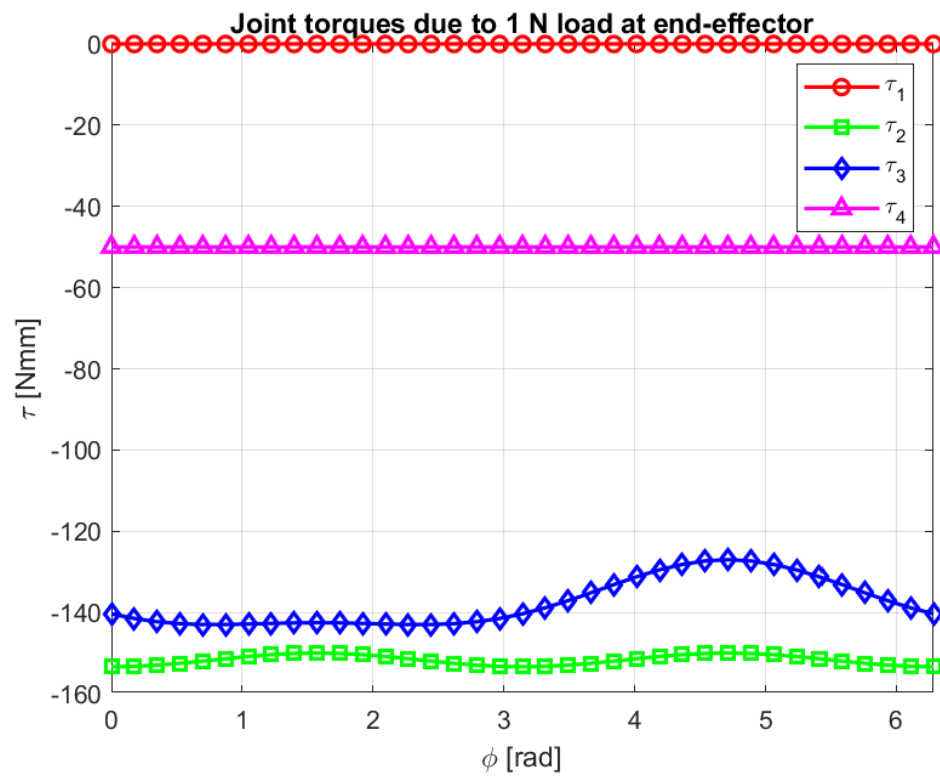


Figure 3.2: Joint torques due to 1N load

Chapter 4

Part 4: Dynamics

Contents

4.1	Problem 10: Inverse Dynamics	25
-----	--	----

4.1 Problem 10: Inverse Dynamics

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

Provide a rough but realistic estimate for I_0 based on the dimensions and mass (e.g. of link 1) and use this estimated value for the remaining links. Then derive the dynamic model of the robot arm in its standard form:

$$\boldsymbol{\tau} = \mathbf{D}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}),$$

and plot the required joint torques τ for the trajectory from Problem 6 or 7.

4.1.1 Euler–Lagrange equations of motion

Following the Euler–Lagrange principle, the equations of motion for each generalized coordinate q_k are

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_k} \right) - \frac{\partial \mathcal{L}}{\partial q_k} = \tau_k,$$

where the Lagrangian $\mathcal{L}$ is the difference between kinetic and potential energy. The kinetic energy is expressed using the general inertia matrix $D(q)$, which contains both translational and rotational inertia terms; the potential energy accounts only for gravitational potential energy.

$$\mathcal{L}(q, \dot{q}) = \mathcal{K} - \mathcal{P} = \frac{1}{2} \dot{q}^T D(q) \dot{q} - P(q) = \sum_{i,j} \frac{1}{2} d_{ij}(q) \dot{q}_i \dot{q}_j - \sum_i m_i g^T r_{c_i}(q).$$

Applying the Euler–Lagrange equations yields, after grouping terms,

$$\sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \sum_{i,j} c_{ijk}(q) \dot{q}_i \dot{q}_j + g_k(q) = \tau_k,$$

or in matrix form

$$\boldsymbol{\tau} = \mathbf{D}(q) \ddot{\mathbf{q}} + \mathbf{C}(q, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(q).$$

To compute the joint torques one must determine the inertia matrix $\mathbf{D}(q)$, the Coriolis/centrifugal matrix $\mathbf{C}(q, \dot{\mathbf{q}})$ and the gravity vector $\mathbf{g}(q)$.

General Inertia Matrix

The general inertia matrix is given by

$$\mathbf{D}(q) = \sum_{i=1}^n \left(m_i \mathbf{J}_{v,c_i}^T \mathbf{J}_{v,c_i} + \mathbf{J}_{\omega_i}^T \mathbf{R}_{i-1} \mathbf{I}_i \mathbf{R}_{i-1}^T \mathbf{J}_{\omega_i} \right).$$

The masses m_i are known, the rotational inertia tensor $\mathbf{I}_i$ can be estimated, and the Jacobians $\mathbf{J}_{v,c_i}$ and $\mathbf{J}_{\omega_i}$ are obtained from the kinematics. The inertia of each robot link is approximated using one main inertia value, denoted I_0 . The scalar is estimated as the principal moments of inertia of link 1 around the x axis. The other moments of inertia are obtained by scaling according to the link's geometric proportions. The same baseline, with appropriate scaling factors, is then applied to the other links. To estimate the rotational inertia constant I_0 , consider link 1 with mass $m_1 = 60$ g and approximate dimensions inferred from Figure 4.1:

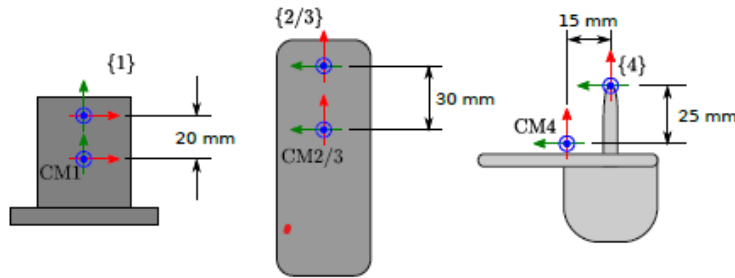


Figure 4.1: Centers of mass and principal axes of inertia for links 1–4.

$$L_x = 30 \text{ mm}, \quad L_y = 40 \text{ mm}, \quad L_z = 50 \text{ mm}.$$

Approximating the link as a rectangular solid, its principal moments of inertia are

$$I_x = \frac{1}{12} m_1 (L_y^2 + L_z^2)$$

An equivalent inertia constant is then taken as

$$I_0 = I_x,$$

which provides an approximation of the inertia contributions in the principal directions indicated in Figure 4.1. The same value I_0 is used for the remaining links as requested.

Based on the problem statement, the inertia matrices in the respective COM frames are

$$I_1 = \begin{bmatrix} I_0 & 0 & 0 \\ 0 & 0.4I_0 & 0 \\ 0 & 0 & 0.9I_0 \end{bmatrix}, \quad I_2 = I_3 = \begin{bmatrix} 0.45I_0 & 0 & 0 \\ 0 & 1.4I_0 & 0 \\ 0 & 0 & 1.2I_0 \end{bmatrix},$$

$$I_4 = \begin{bmatrix} 0.5I_0 & 0 & 0 \\ 0 & 0.5I_0 & 0 \\ 0 & 0 & 0.5I_0 \end{bmatrix}.$$

The Jacobians are expressed with respect to the COM of each link. For link i the COM linear velocity and angular velocity follow from

$$\dot{\mathbf{o}}_{c,i} = J_v^{(i)}(q) \dot{q}, \quad \boldsymbol{\omega}_i = J_\omega^{(i)}(q) \dot{q}.$$

For revolute joints the Jacobian columns are

$$J_{v,j}^{(i)} = \mathbf{z}_j \times (\mathbf{o}_{c,i} - \mathbf{o}_j), \quad J_{\omega,j}^{(i)} = \mathbf{z}_j,$$

where joint origins $\mathbf{o}_j$ and axes $\mathbf{z}_j$ are obtained from the homogeneous transforms $T_{0j}(q)$. The COM of each link is given in its local frame and mapped to the base frame by

$$\mathbf{o}_{c,i} = \mathbf{o}_i + R_i^0(q) \mathbf{r}_{c,i}.$$

Combining these Jacobians with the estimated inertia tensors and masses yields the full inertia matrix $D(q)$.

4.1.2 Coriolis matrix

The Coriolis/centrifugal matrix follows from the Christoffel symbols:

$$[C(q, \dot{q})]_{kj} = \sum_{i=1}^n c_{ijk}(q) \dot{q}_i = \sum_{i=1}^n \frac{1}{2} \left(\frac{\partial d_{kj}}{\partial q_i} + \frac{\partial d_{ki}}{\partial q_j} - \frac{\partial d_{ij}}{\partial q_k} \right) \dot{q}_i,$$

which can be computed directly from the inertia matrix $D(q)$.

Potential energy vector

The gravity terms are obtained from the potential energy using the COM heights:

$$g(q) = \begin{bmatrix} g_1(q) \\ \vdots \\ g_n(q) \end{bmatrix}, \quad g_k(q) = \frac{\partial P}{\partial q_k}.$$

Torque Computation for a Given Trajectory

Finally, the required joint torques are computed using

$$\tau = D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q),$$

evaluated along the trajectory from Problem 6 or Problem 7.

The results can be seen in Fig. 4.2. This figure shows the joint torques required to follow the trajectory shown in Fig. 2.2. Note the similarity to Fig. 3.2. Motors 2 and 3 take over the task of holding the arm up against gravity.

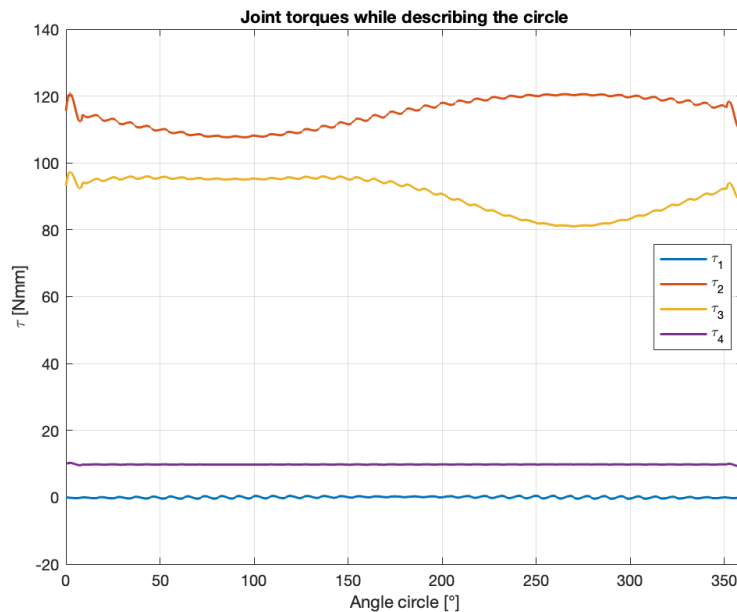


Figure 4.2: Torques required for following path shown in 2.2.

Chapter 5

Part 5: Computer Vision and Control

Contents

5.1	Problem 11: Real World Implementation	29
5.1.1	Computer Vision	29
5.1.2	Image to Real World Coordinates: Backwards Projection	29
5.1.3	Implementation in Code	32
5.1.4	Results	32

5.1 Problem 11: Real World Implementation

Botond: 20%, Emma: 20%, Henrik: 20%, Jente: 20%, Theo: 20%

NOTE: The group chose variant b.

Create a software which enables the robot arm to recognize all red "smarties" among a cluster of colorful "smarties" on a surface in front of it, and to probe all of them with its stylus. You must provide details about the theory behind all necessary calculations, the code implementation (in Matlab or Python) and a video recording demonstrating how well the robot is performing the desired task.

5.1.1 The Robot Arm

The revolute joints of the Robot Arm as seen on Fig. 1.1 are actuated by Dynamixel AX-12A servo motors [2]. These come with an SDK (software development kit) that contains low-level driver functions that enable communication to it through its USB port.

5.1.2 Computer Vision

The goal is to point the robot stylus at red smarties, that are placed among other coloured smarties in front of it. For this the red smarties, which are essentially red circles,

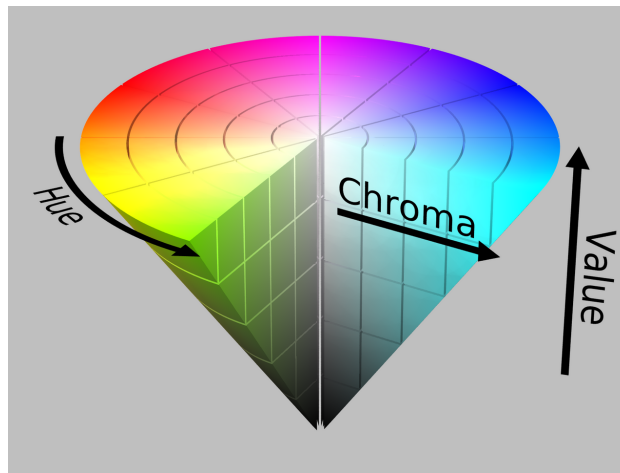


Figure 5.1: HSV color space; Adapted from here.

have to be detected. This is a classical problem in computer vision and well known. The problem can be structured in two steps:

1. Filter out non-red smarties
2. Get pixel coordinates of red areas/circles

Filtering out the non-red smarties

Fig. 5.2 shows an example view of the robot camera. Detecting areas/circles in this one would also detect non-red smarties. However, the goal is to only detect the red ones. The other colors can be filtered out using thresholding. This thresholding is done best in the HSV-color space. This is preferred, because in the HSV color space color can be encoded more robustly against lighting conditions. The hue value measured in degrees (refer to Fig. 5.1) can effectively be used to determine the color of a pixel "qualitatively". Creating a mask can be done using bitwise operators and the hue range of red. For numeric values see Tab. 5.1.

Post Processing As some post processing to account to make it even more robust against different lighting conditions, *opening* is performed on the image. First the image is eroded - effectively removing noisy patches - and then dilated to close holes that have emerged and scale the areas up again.

Get pixel coordinates of non-red smarties

Now the pixel coordinates of the red areas have to be found. Theoretically, circles are best found using the Hough-transform. However, the smarties are not perfectly circular and due to the thresholding the edges may become rougher. This project has found a Hough transform based detection to not be robust enough to find all the red smartie centers. Hence, a more robust way is needed. Here an advantage can be taken from the fact that the

Table 5.1: Isolation of red smarties including opening

Variable	Value / Description
lower_red1	0, 100, 100
upper_red1	20, 255, 255
lower_red2	150, 100, 100
upper_red2	179, 255, 255
mask1	<code>cv2.inRange(hsv, lower_red1, upper_red1)</code>
mask2	<code>cv2.inRange(hsv, lower_red2, upper_red2)</code>
red_mask	<code>cv2.bitwise_or(mask1, mask2)</code>
kernel1	<code>np.ones((4, 4), np.uint8)</code>
kernel2	<code>np.ones((16, 16), np.uint8)</code>
red_mask	<code>cv2.erode(red_mask, kernel1)</code>
red_mask	<code>cv2.dilate(red_mask, kernel2)</code>



Figure 5.2: Example view of robot

thresholded image is a binary image. This allows the use of the algorithm of [4], which is implemented in Open-CV's *findContours* method, which is based on border following and, hence, more robust to non-circular areas.

Having the contours found, the center of these can be determined by finding the circle of minimal radius that encloses each contour. This can conveniently be done using an iterative algorithm, implemented in Open-CV's *minEnclosingCircle* method.

The result of all these operations can be seen in Fig. 5.3.

5.1.3 Image to Real World Coordinates: Backwards Projection

The output from the computer vision functions will be pixel coordinates. These can be related to real world coordinates using either stereo vision, images of different poses or known constraints of location. The latter is applicable to this problem. As the ring or rings will be placed in front of the robot the height/the plane at which they are located in the world coordinate frame is known. This is a very useful constraint, as from the pixel coordinates a line on which the circle center must lie can be obtained. Finding the intersection of that line

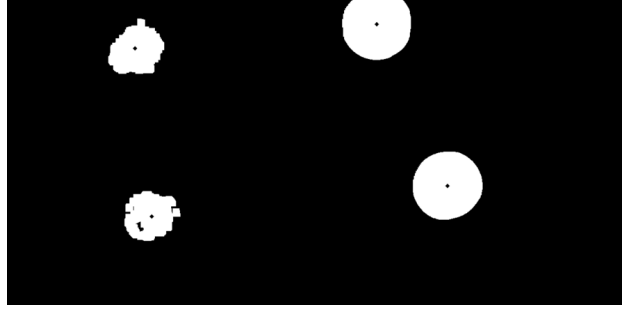


Figure 5.3: Thresholded Image and detections (black dots)

with the known plane, on which the circle must lie, allows to compute the rings location in the world frame.

From Image Coordinates to World Frame Coordinates Assuming a linear camera model, the homogeneous image coordinates can be obtained:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K x_{\circ}^5 \quad (5.1)$$

Where $x_{\circ}^5$ are the coordinates of the circle center in the camera frame and K is the camera matrix entailing the camera intrinsics. s is a scale factor, sometimes also called λ . This indicates that the projection is only defined up to a scale.

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (5.2)$$

Consequently, the world coordinates X , Y and Z of the circle will have the following projection, where R_0^5 and t_0^5 describe the world coordinate frame's rotation and translation with respect to the camera frame, respectively and can be obtained from the inverse of the previously obtained T_5^0 (Which will be modified in a little to match conventions).

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R_0^5 | t_0^5] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (5.3)$$

Consequently,

$$K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{x_{\circ}^5}{s} = r_c \quad (5.4)$$

gives a scaled version of the vector pointing at the circle center in the camera frame. R_5^0 can

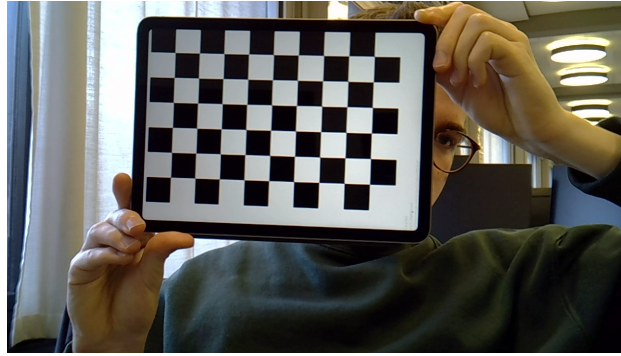


Figure 5.4: Sample Calibration Image

be used to get that vector in the world frame.

$$r_w = R_5^0 r_c \quad (5.5)$$

The ray on which the ring center then may lie can be described by:

$$R(s) = o_5^0 + s \cdot r_w \quad (5.6)$$

$$R_z(s) = o_{5z}^0 + s \cdot r_{wz} \quad (5.7)$$

o_5^0 can be obtained from T_5^0 . As the z -coordinate/the plane upon which the ring lies is known, Eq. 5.7 can be solved for s , which then can be used in Eq. 5.6 to compute the x and y components corresponding to z .

Camera Calibration - Obtaining Camera Matrix K

In real world camera calibration the camera matrix K can be obtained by sampling points of a known relative position and scale, such as the checker board shown in Fig. 5.4, with a fixed box size (20mm in this case). An optimization algorithm can then solve for the projection matrix, minimizing the error over several of such images like in the figure.

Matlab offers a camera calibration app, which was used to obtain the camera matrix K . The group also did a calibration using OpenCV. Both approaches resulted in almost identical matrices.

$$K = \begin{bmatrix} 1.4622 \times 10^3 & 0 & 657.6263 \\ 0 & 1.4617 \times 10^3 & 349.7438 \\ 0 & 0 & 1 \end{bmatrix} \quad (5.8)$$

Fig. 5.5 shows the projection using that matrix and Fig. 5.6 shows the mean reprojection error in pixels per image. These results were deemed acceptable.

However, when looking at Fig. 5.5, one sees that the camera frame is rotated differently from frame 5 from part 1.

Hence a new formulation of frame 5 is found that accounts for that orientation convention and additionally accounts for the additional translation sideways of the stylus, which

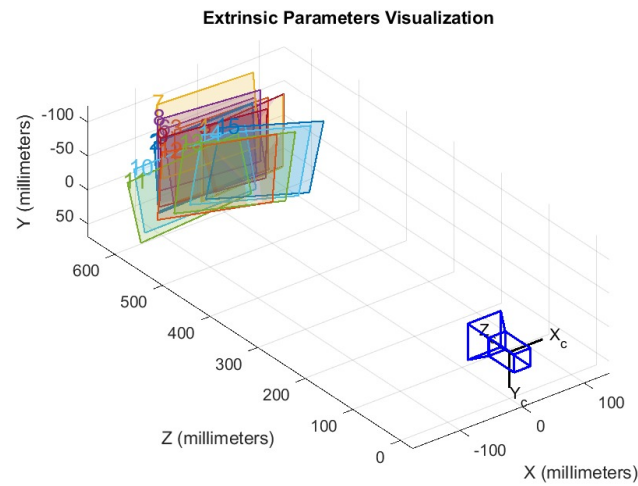


Figure 5.5: Calibration images in camera frame



Figure 5.6: Mean Projection Error

has not been accounted for in Fig.1.1.

$$T_{5_{new}}^0 = T_4^0 * \begin{bmatrix} & & -10 \\ R_5^4 & 25 \\ & 15 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.9)$$

Where

$$R_5^4 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad (5.10)$$

This now accounts for the actual displacement (in *mm*) and orientation of the camera frame with respect to the stylus.

5.1.4 Implementation in Code

Python was chosen as the programming language due to the Dynamixel SDK providing low-level support for it with a library, as well as libraries like NumPy that make working with matrices and mathematics easier. Most group members are also familiar with it which made debugging and development easier. The codebase can be broken up into 4 main parts: main control loop, kinematics, computer vision, servo driver. The code is loosely object oriented, since this approach provides sensible encapsulation possibilities for parts like the kinematics or the servo driver where each function and variable is closely related to one-another. However, the computer vision functions remain un-encapsulated as calibration happens only once and it is not worth the structural overhead to merge it into a whole class. The Smartie finding functions are independent of the calibration and only the intrinsic and the extrinsic matrices are used.

Main control loop

The state machine responsible for fulfilling the task is illustrated on Fig. 5.7.

Kinematics

To encapsulate the physical parameters of the arm as well as relevant functions such as **forward** and **inverse** which calculate the forward and backwards kinematics respectively. These combined with the driver functions provide an easy method of moving the robot arm to an arbitrary position. For further discussion on the math behind these functions refer to Section 1 and 1.1.2.

Computer vision

The robot arm uses a camera as a sensor and to access the video feed OpenCV provides an easy to use function called **VideoCapture**. This allows the capturing of individual frames that can be used for camera calibration or the main control loop for finding Smarties. To

calibrate the camera OpenCV's `findChessboardCornersSB` function was used. This provides robust corner detection used to make the image and object point correspondences that can be passed into the `calibrateCamera` function to get the camera matrix and distortion vector. On the calculation of the 3D position of each Smartie refer to Section 5.1.3.

Servo driver

Controlling the servo motors is as simple as writing specific values into its registers. With the help of the register map [1] the function and address of each register is known. For example to move a servo to a specific angle the following function can be used:

```
packetHandler.write2ByteTxRx(portHandler, joint_id, goal_pos_addr, joint_angle)
```

where `packetHandler`, all of its functions and `portHandler` are provided by the DynamixelSDK, `joint_id` is the index of the servo that the command writes to, `goal_pos_addr` is the register offset or address to write to, and `joint_angle` is the converted angle to move to.

To convert between radians and the 10-bit numerical value representing the operating range of the servo the following formula is used:

$$\text{joint_angle} = \frac{\text{angle}}{\frac{5\pi}{3}} * 1024 \quad (5.11)$$

where *angle* is the desired position, $\frac{5\pi}{3}$ is the max operating range of the servo in radians, and 1024 is the 10-bit range of the servo motor.

By sending a command to each of the servos with a goal position the desired pose can be obtained. A wrapper function does this with an added safety feature that clamps out-of-range angles to the max or the min position of each motor individually.

5.1.5 Results

A video demonstration can be accessed with this link: <https://youtu.be/sh76rIyBgE0> Alternatively, click here.

The video shows the robot executing the task from two perspectives. The robot reliably finds the red smarties with the computer vision pipeline described earlier and points on them. There is some inaccuracy in the height the stylus hovers over the smarties, but it is tolerable and can be explained by calibration inaccuracies of the camera calibration and when finding the motor center positions and actual dimensions of the robot arm. The group is very pleased with the result and found it a valuable learning experience.

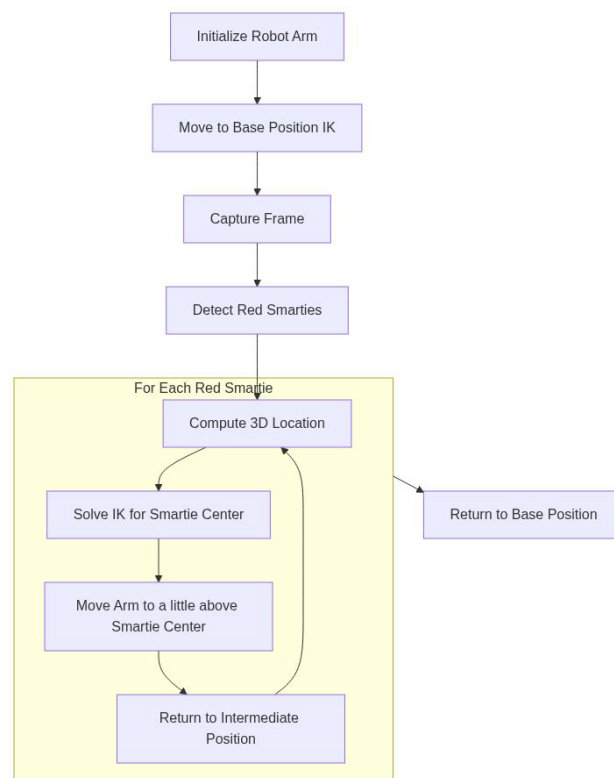


Figure 5.7: The SMARTie-Algorithm

Chapter 6

Signatures & Formalities

6.1 Contributions

All team members contributed equally to this project:

- Botond Bencsik - s254257 $\Rightarrow$ 20%
- Emma Royant - s253783 $\Rightarrow$ 20%
- Henrik Maarten Bongers - s253818 $\Rightarrow$ 20%
- Jente Staelens - s252877 $\Rightarrow$ 20%
- Theo Haes - s253792 $\Rightarrow$ 20%

6.2 Signatures



Figure 6.1: Bencsik Botond



Figure 6.2: Emma Royant

A handwritten signature in black ink, reading 'Bongers' in a cursive style.

Figure 6.3: Henrik Maarten Bongers

A handwritten signature in black ink, reading 'Jente Staelens' in a cursive style.

Figure 6.4: Jente Staelens

A handwritten signature in black ink, reading 'Theo Haes' in a cursive style.

Figure 6.5: Theo Haes

6.3 Use of AI

Generative AI tools such as ChatGPT and Microsoft Copilot were used for occasional debugging, to look up syntax and plotting, as well as, visualization scripts. However, all the solutions were worked out by members of the group.

Bibliography

- [1] Robotis. Robotis dynamixel ax-12a register map. URL: <https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/#control-table-data-address>.
- [2] Robotis. Robotis dynamixel ax-12a servo motor's website. URL: <https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/>.
- [3] Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Modeling and Control, 1st Ed.* John Wiley & Sons, Inc., 2006.
- [4] Satoshi Suzuki and Keiichi Abe. Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1):32–46, 1985. URL: <https://www.sciencedirect.com/science/article/pii/0734189X85900167>, doi:10.1016/0734-189X(85)90016-7.

Appendix A

Links

A.1 Code

https://github.com/royantemma/DTU_Robotics_Project_Assignement_Group20/tree/main/RoboControl

A.2 Video of Results

<https://youtu.be/sh76rIyBgE0>

Appendix B

Tables & Numerical Results

B.1 Joint configurations for circular trajectory, in $^{\circ}$

	θ_1	θ_2	θ_3	θ_4
0	12.042575	81.942843	-95.678372	13.735530
1	11.864845	82.719772	-93.060418	10.340646
2	11.335700	83.399151	-90.464340	7.065189
3	10.467483	83.972856	-87.971972	3.999116
4	9.281392	84.439629	-85.667701	1.228072
5	7.808158	84.803569	-83.635610	-1.167959
6	6.088528	85.072351	-81.955560	-3.116791
7	4.173151	85.255206	-80.698239	-4.556967
8	2.121549	85.360792	-79.919660	-5.441132
9	0.000000	85.395249	-79.655965	-5.739284
10	-2.121549	85.360792	-79.919660	-5.441132
11	-4.173151	85.255206	-80.698239	-4.556967
12	-6.088528	85.072351	-81.955560	-3.116791
13	-7.808158	84.803569	-83.635610	-1.167959
14	-9.281392	84.439629	-85.667701	1.228072
15	-10.467483	83.972856	-87.971972	3.999116
16	-11.335700	83.399151	-90.464340	7.065189
17	-11.864845	82.719772	-93.060418	10.340646
18	-12.042575	81.942843	-95.678372	13.735530
19	-11.864845	81.084550	-98.240879	17.156329
20	-11.335700	80.169930	-100.676490	20.506559
21	-10.467483	79.233013	-102.920632	23.687618
22	-9.281392	78.316003	-104.916412	26.600409
23	-7.808158	77.467159	-106.615262	29.148104
24	-6.088528	76.737195	-107.977398	31.240203
25	-4.173151	76.174357	-108.972049	32.797692

26	-2.121549	75.818777	-109.577451	33.758674
27	-0.000000	75.697131	-109.780679	34.083549
28	2.121549	75.818777	-109.577451	33.758674
29	4.173151	76.174357	-108.972049	32.797692
30	6.088528	76.737195	-107.977398	31.240203
31	7.808158	77.467159	-106.615262	29.148104
32	9.281392	78.316003	-104.916412	26.600409
33	10.467483	79.233013	-102.920632	23.687618
34	11.335700	80.169930	-100.676490	20.506559
35	11.864845	81.084550	-98.240879	17.156329
36	12.042575	81.942843	-95.678372	13.735530

Appendix C

Code

C.1 Forward and Inverse Kinematics

```
class Dynamixel:
    def __init__(self, Ts=[]):

        self.theta_1, self.theta_2, self.theta_3, self.theta_4 = sp.symbols('
        theta_1_theta_2_theta_3_theta_4')
        # Save individual transforms
        self.Ts = Ts

        # Compute full transform to end-effector
        self.T_0_ee = sp.eye(4)
        for T in Ts:
            self.T_0_ee *= T
        self.T_0_ee = sp.simplify(self.T_0_ee)

        # Compute transforms from base to each intermediate frame
        self.T_0_is = []
        T_accum = sp.eye(4)
        for T in Ts:
            T_accum *= T
            self.T_0_is.append(sp.simplify(T_accum))

        self.d1 = 0.050 # m
        self.a2 = 0.093
        self.a3 = 0.093
        self.a4 = 0.050

    def forward(self,
```

```

        joints=[], # list of (joint_symbol, value)
        doallframes=True,
        verbose=False):

    # Build substitution dictionary
    subs_dict = {q: val for q, val in joints}

    # Substitute into end-effector transform
    T_0_ee = self.T_0_ee.subs(subs_dict)
    # Convert to NumPy
    T_0_ee = np.array(T_0_ee.evalf().tolist()).astype(np.float64)
    self.ee_pos = T_0_ee[0:3,3]

    if verbose:
        print("FK: _The_EE_is_at:", self.ee_pos)

    # Substitute into intermediate frames
    T_0_is = []
    if doallframes:
        for T in self.T_0_is:
            T_num = T.subs(subs_dict).evalf()
            T_num = np.array(T_num.tolist()).astype(np.float64)
            T_0_is.append(T_num)

    if doallframes:
        return T_0_is
    else:
        return T_0_ee

def inverse(self, x, y, z, x_04 = None, theta_ee=-np.pi/4, verbose=False):
    theta_1 = np.arctan2(y, x)

    if x_04 is not None:
        self.theta_ee = np.arcsin(x_04/self.a4)
        theta_ee = self.theta_ee
    else:
        x_04 = self.a4 * np.sin(theta_ee)

    r_ee = np.sqrt(x**2+y**2)

    r = r_ee - self.a4 * np.cos(theta_ee)

    z_c = z + x_04

```

```

s = z - self.d1 - self.a4*np.sin(theta_ee)
# Elbow up config is applied!
cos_t3 = -(self.a2**2 + self.a3**2 - r**2 - s**2) / (2 * self.a2 * s)
sin_t3 = np.sqrt(1-cos_t3**2)
theta_3 = np.arctan2(-sin_t3, cos_t3)
theta_2 = np.arctan2(s, r) + np.arctan2(self.a3*sin_t3, self.a2+self.a4*np.sin(theta_3))
theta_4 = -theta_2 - theta_3 + theta_ee

if verbose:
    print("IK:_Determined_angles",
          [(self.theta_1, np.rad2deg(theta_1)),
           (self.theta_2, np.rad2deg(theta_2)),
           (self.theta_3, np.rad2deg(theta_3)),
           (self.theta_4, np.rad2deg(theta_4))])

return [
    (self.theta_1, theta_1),
    (self.theta_2, theta_2),
    (self.theta_3, theta_3),
    (self.theta_4, theta_4)
]
```